Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Систем Управления и Робототехники

Лабораторная работа №5

по Программированию

Вариант 473

Выполнил:

Рязанов Даниил Дмитриевич

Группа R3140

Преподаватель:

Вальц Мартин Эдуардович

Санкт-Петербург 2025

Содержание

Текст задания ***3***

Программная архитектура ***5***

package core **5**

package dragon **5**

package dragon.view 5

package collection **6**

package storage **7**

package ui **7**

Main **8**

Диаграмма классов **8**

Упаковка и запуск программы ***8***

Выводы по работе ***8***

Приложение ***9***

Main **9**

package core **11**

BadDataException 11

Coordinates 12

Defaults 13

package dragon **14**

Dragon 14

DragonHead 19

DragonType 19

package dragon.view **20**

View 20

JsonView 21

package collection *API* **27**

Collection 29

package storage **32**

Storage 32

FileStorage 32

ui **35**

Command 35

Commands 35

Executable 36

TextUIHandler 36

UI 50

Текст задания

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса Dragon, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа java.util.HashSet
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: **переменная окружения**.
- Данные должны храниться в файле в формате json
- Чтение данных из файла необходимо реализовать с помощью класса java.io.InputStreamReader
- Запись данных в файл необходимо реализовать с помощью класса java.io.BufferedWriter
- Все классы в программе должны быть задокументированы в формате javadoc.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутсвие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- help : вывести справку по доступным командам
- info : вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- show : вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- add {element} : добавить новый элемент в коллекцию
- update id {element} : обновить значение элемента коллекции, id которого равен заданному
- remove_by_id id : удалить элемент из коллекции по его id
- clear : очистить коллекцию
- save : сохранить коллекцию в файл
- execute_script file_name : считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- exit : завершить программу (без сохранения в файл)
- remove_greater {element} : удалить из коллекции все элементы, превышающие заданный
- remove_lower {element} : удалить из коллекции все элементы, меньшие, чем заданный
- history : вывести последние 8 команд (без их аргументов)
- count_by_type type : вывести количество элементов, значение поля type которых равно заданному
- count_greater_than_type type : вывести количество элементов, значение поля type которых больше заданного

- filter_starts_with_name name : вывести элементы, значение поля name которых начинается с заданной подстроки

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, String, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является enum'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'е; введена строка вместо числа; введённое число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Dragon {
    private Long id; //Поле не может быть null, Значение поля должно быть больше 0,
Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться
автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDateTime creationDate; //Поле не может быть null, Значение
этого поля должно генерироваться автоматически
    private int age; //Значение поля должно быть больше 0
    private long weight; //Значение поля должно быть больше 0
    private boolean speaking;
    private DragonType type; //Поле может быть null
    private DragonHead head;
}
public class Coordinates {
    private Float x; //Значение поля должно быть больше -359, Поле не может быть null
    private Float y; //Максимальное значение поля: 603, Поле не может быть null
}
public class DragonHead {
    private float size;
    private float toothCount;
}
public enum DragonType {
    WATER,
    UNDERGROUND,
    AIR,
    FIRE;
}
```

Программная архитектура

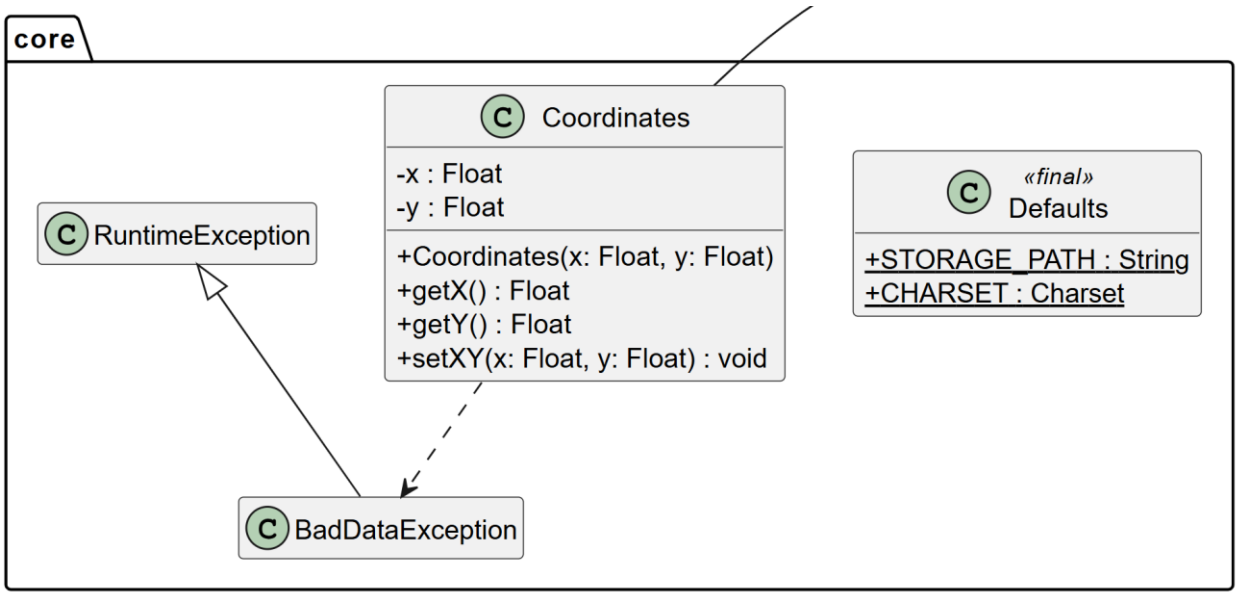
Все иходники программы расположены в Github-репозитории MrRyabena/DragonsDB.

package core

Coordinates: value-object координат (x,y) с валидацией ограничений; выбрасывает BadDataException при некорректных значениях.

BadDataException: единый тип ошибки для нарушений ограничений модели/ввода.

Defaults: централизованные константы приложения (путь по умолчанию к файлу, кодировка).



package dragon

Dragon: инкапсулирует данные о драконе (id, имя, координаты, дата создания, возраст, вес, флаги/атрибуты).

DragonHead: часть доменной модели, описывает параметры головы дракона (размер, кол-во зубов) и используется как поле в Dragon.

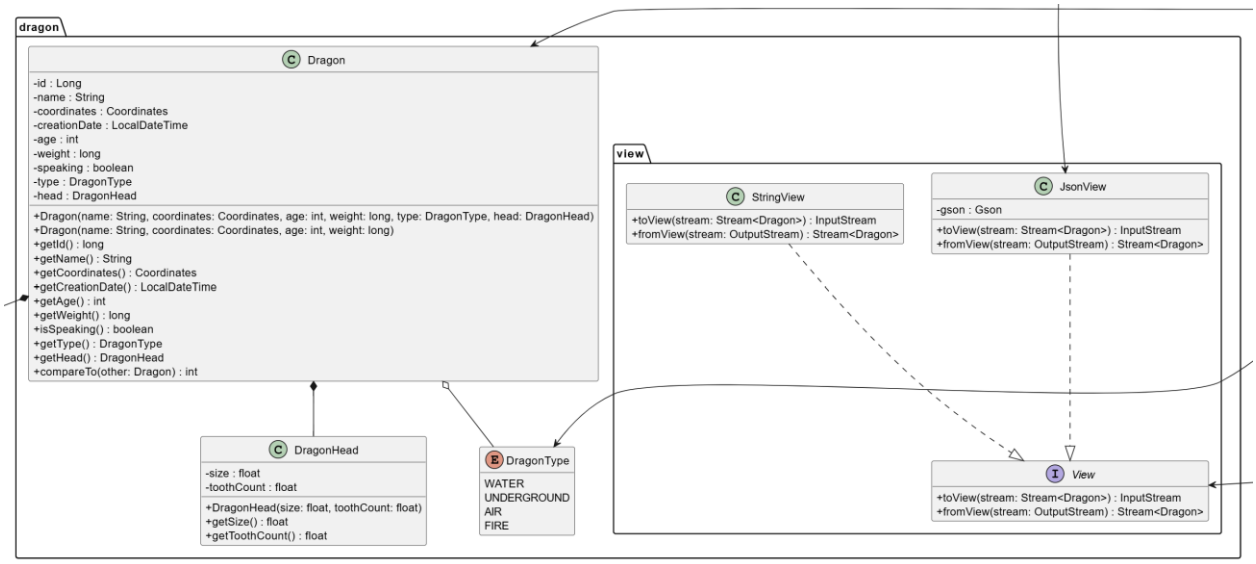
DragonType: перечисление типов драконов, используется для фильтраций/подсчётов и как атрибут Dragon.

package dragon.view

View: интерфейс представления/преобразования потока Stream<Dragon> в I/O-форму и обратно (кодирование/декодирование).

JsonView: реализация View на Gson; сериализует/десериализует коллекцию как JSON-массив, содержит обработку ошибок парсинга (битый JSON не нарушает работу приложения).

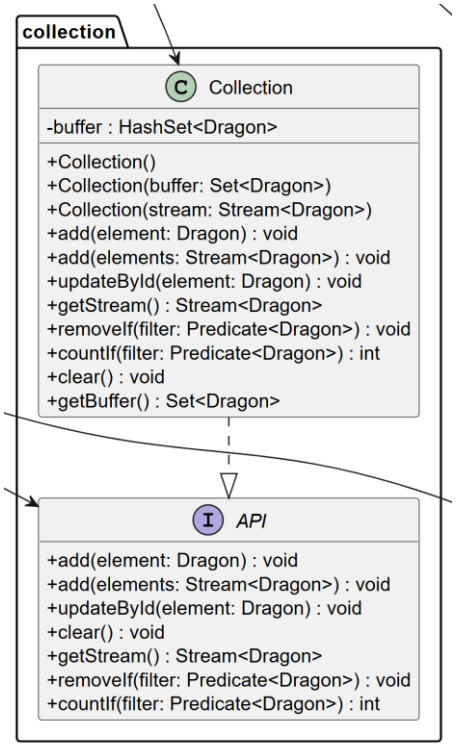
StringView: текстовое представление для вывода пользователю (читаемый многострочный формат); обратный парсинг не поддерживается.

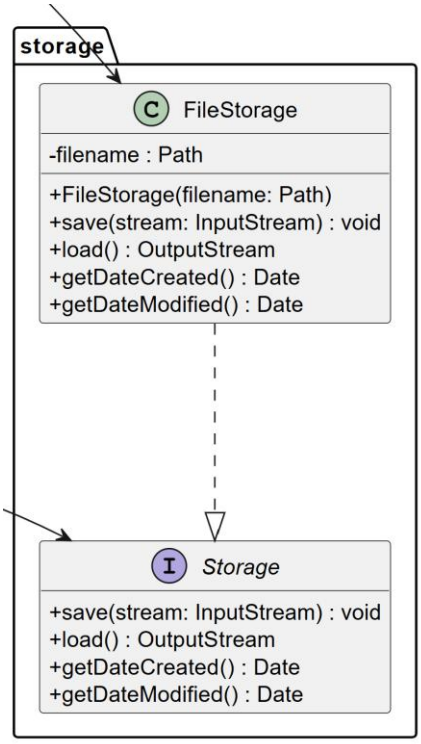


package collection

API: интерфейс работы с коллекцией Dragon по парадигме CRUD (add/update/remove/clear, stream-выдача, count/remove по предикату).

Collection: in-memory реализация API на основе HashSet<Dragon>; даёт поток элементов для отображения/сохранения и поддерживает операции удаления/подсчёта по условиям.





package storage

Storage: абстракция персистентности (save/load) и метаданных ресурса (дата создания/изменения).

FileStorage: файловая реализация Storage; хранит сериализованное содержимое в одном файле и берёт метаданные из файловой системы.

package ui

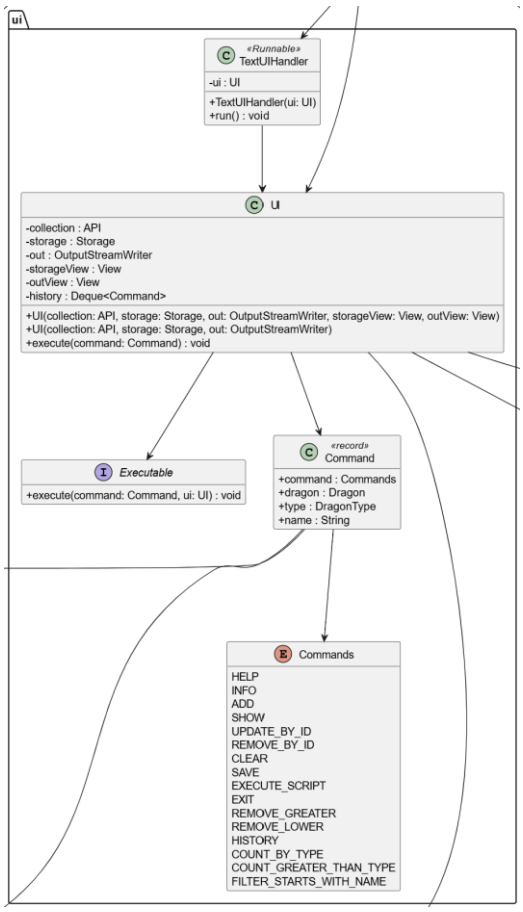
Commands: перечисление поддерживаемых команд CLI (текст + вычисляемый code для диспетчеризации).

Command: DTO/record результата парсинга команды (тип команды + аргументы: Dragon, DragonType, строка).

Executable: функциональный интерфейс обработчика команды.

UI: фасад приложения: связывает collection.API, storage.Storage и View; содержит реестр обработчиков команд и историю последних команд; выполняет команды и инициирует сохранение/вывод.

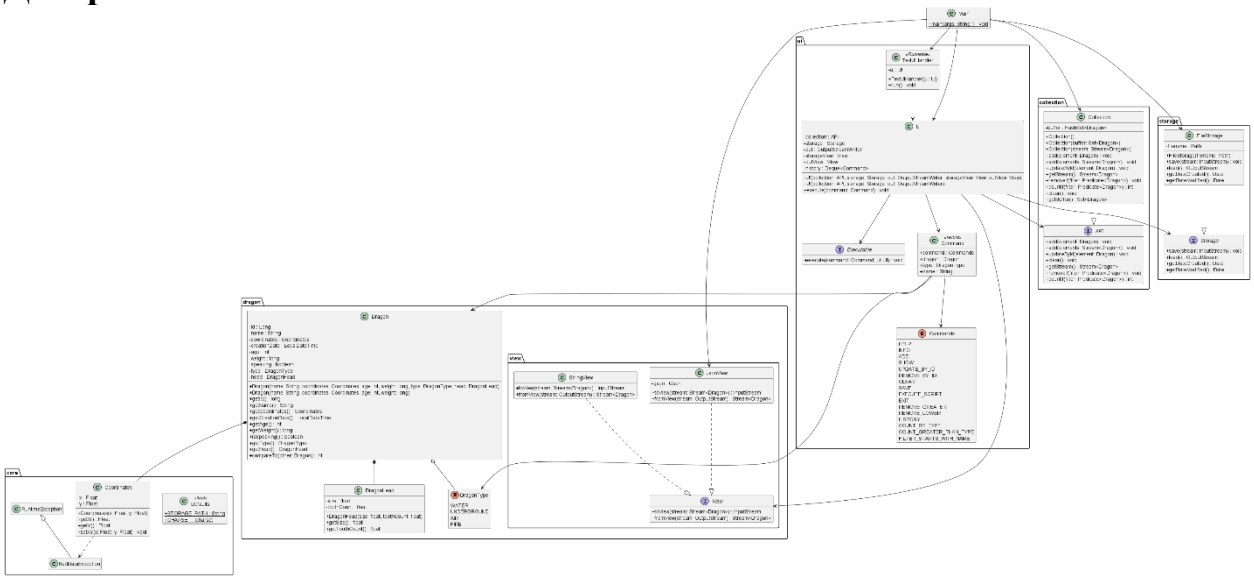
TextUIHandler: читает команды из stdin и скриптов, валидирует ввод, собирает Dragon по полям, поддерживает execute_script с ограничением глубины и защитой от рекурсии.



Main

Точка входа: поднимает зависимости (storage + view + коллекция + UI), загружает данные из файла и запускает TextUIHandler для интерактивной работы.

Диаграмма классов



Упаковка и запуск программы

Исходный код программы расположен в папке src.
Для сборки, компиляции и упаковки программы используется утилита Maven. Для корректной работы Gson с классами времени, необходимо открыть библиотеке их область видимости. Это прописано в pom.xml и требуется при запуске программы:

```
mvn clean

mvn package -DskipTests

java --add-opens java.base/java.time=ALL-UNNAMED -jar target/DragonsDB.jar
```

Выводы по работе

Цель работы достигнута, разработано консольное приложение для интерактивного управления коллекцией объектов Dragon с поддержкой полного набора команд из задания (просмотр, добавление, обновление по id, удаление, очистка, сохранение, выполнение скриптов, фильтрации и подсчёты).

Требования к модели данных выполнены, реализованы классы Dragon, Coordinates, DragonHead и перечисление DragonType; ограничения полей обеспечены проверками, а поля, генерируемые автоматически (id, creationDate), формируются внутри Dragon.

Хранение объектов организовано в java.util.HashSet, что соответствует требованиям задания и обеспечивает уникальность элементов по логике набора.

Реализована персистентность в JSON: данные коллекции сохраняются и загружаются из файла в формате JSON. Чтение/запись выполнены через требуемые классы ввода-вывода (InputStreamReader, BufferedWriter).

Реализованы автозагрузка при старте и настройка через переменную окружения. При запуске приложение получает путь к файлу из переменной окружения (или использует значение по умолчанию из Defaults), после чего автоматически загружает коллекцию и переходит в интерактивный режим.

Предусмотрена обработка некорректного пользовательского ввода (повторный ввод, сообщения об ошибке) и ошибок чтения/парсинга JSON (битый файл не приводит к аварийному завершению, выводится диагностическое сообщение).

Архитектура проекта структурирована по слоям: код разделён на пакеты по ответственности (core, dragon, dragon.view, collection, storage, ui), что улучшает читаемость, расширяемость и изоляцию изменений (например, можно заменить формат хранения или способ представления, не затрагивая логику UI и коллекции).

Код задокументирован, классы и ключевые элементы программы снабжены документацией в формате Javadoc, что облегчает сопровождение и соответствует формальным требованиям лабораторной.

В ходе данной лабораторной работы я научился разрабатывать приложение, демонстрирующее применение принципов и паттернов ООП, работу с коллекциями Java, потоками ввода-вывода, сериализацией в JSON и организацию интерактивного CLI с поддержкой сценариев.

Приложение

Main

```
import java.io.OutputStreamWriter;
import java.nio.file.InvalidPathException;

import dragon.view.JsonView;
import storage.FileStorage;
import ui.TextUIHandler;
import ui.UI;

/**
 * Application entry point.
 *
 * <p>Bootstraps storage, loads the initial collection state, and starts the in-
teractive text UI.
 */
public class Main {
```

```java
    /**
     * Starts the application.
     *
     * @param args command-line arguments (unused)
     */
    public static void main(String args[]) {

        String storage_path = null;
        collection.Collection collection = new collection.Collection();

        try {
            storage_path = System.getenv("STORAGE_PATH");
        } catch (InvalidPathException e) {
            e.printStackTrace();
        }

        if (storage_path == null) {
            storage_path = core.Defaults.STORAGE_PATH;
            System.err.printf(
                    "Environment variable STORAGE_PATH not found! Using default
path: \"%s\"\n",
                    storage_path);
        }

        var storage = new FileStor-
age(java.nio.file.FileSystems.getDefault().getPath(storage_path));
        var jsonView = new JsonView();

        collection.add(jsonView.fromView(storage.load()));

        var ui = new UI(collection, storage, new OutputStreamWriter(System.out));
        var textUIHandler = new TextUIHandler(ui);

        textUIHandler.run();
    }
}
```

package core

BadDataException

```java
package core;

/**
 * Runtime exception indicating that provided data violates domain constraints.
 */
public class BadDataException extends RuntimeException {
    /** Creates an exception without message. */
    public BadDataException() {
        super();
    }

    /**
     * Creates an exception with a message.
     *
     * @param message error details
     */
    public BadDataException(String message) {
        super(message);
    }

    /**
     * Creates an exception with a message and a root cause.
     *
     * @param message error details
     * @param cause root cause
     */
    public BadDataException(String message, Throwable cause) {
        super(message, cause);
    }

    /**
     * Returns a human-readable message with a fixed prefix.
     *
     * @return formatted error message
     */
    @Override
    public String getMessage() {
        return "Incorrect data: " + super.getMessage();
    }
}
```

```java
package core;

/**
 * 2D coordinates value object with validation constraints.
 */
```

Coordinates

```java
public class Coordinates {

    /**
     * Constructs coordinates with validation.
     *
     * @param x x coordinate (must be greater than -359, cannot be null)
     * @param y y coordinate (must be less than or equal to 603, cannot be null)
     * @throws BadDataException if constraints are violated
     * @throws NullPointerException if x or y is null
     */
    public Coordinates(Float x, Float y) throws BadDataException, NullPointerEx-
ception {
        setXY(x, y);
    }

    /**
     * Returns x coordinate.
     *
     * @return x coordinate
     */
    public Float getX() {
        return x;
    }

    /**
     * Returns y coordinate.
     *
     * @return y coordinate
     */
    public Float getY() {
        return y;
    }

    /**
     * Sets both coordinates with validation.
     *
     * @param x x coordinate (must be greater than -359, cannot be null)
     * @param y y coordinate (must be less than or equal to 603, cannot be null)
     * @throws BadDataException if constraints are violated
     * @throws NullPointerException if x or y is null
     */
    public final void setXY(Float x, Float y) throws BadDataException, Null-
PointerException {
        if (x == null || y == null) throw new NullPointerException();
        if (x <= -359) throw new BadDataException("Value 'x' must be greater than
-359");
        if (y > 603) throw new BadDataException("Value 'y' must be less than
603");
```

```java
        this.x = x;
        this.y = y;
    }

    /** X coordinate. Must be greater than -359, cannot be null. */
    private Float x;

    /** Y coordinate. Maximum value is 603, cannot be null. */
    private Float y;
}
```

Defaults

```java
package core;

import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

/**
 * Application-wide default constants.
 */
public final class Defaults {
    /** Default path used when the environment variable is not provided. */
    public static final String STORAGE_PATH = "data.json";

    /** Charset used for all text I/O operations. */
    public static final Charset CHARSET = StandardCharsets.UTF_8;
}
```

package dragon

Dragon

```java
package dragon;

import core.BadDataException;
import core.Coordinates;

/**
 * Represents a dragon entity with various attributes such as name, coordinates,
age, etc.
 * Implements Comparable for sorting by ID and Serializable for persistence.
 */
public class Dragon implements Comparable<Dragon>, java.io.Serializable {

    /**
     * Constructs a new Dragon with all attributes specified.
     *
     * @param name the name of the dragon (cannot be null or empty)
     * @param coordinates the coordinates of the dragon (cannot be null)
     * @param age the age of the dragon (must be greater than 0)
     * @param weight the weight of the dragon (must be greater than 0)
     * @param type the type of the dragon (can be null)
     * @param head the head of the dragon (cannot be null)
     */
    public Dragon(
            String name,
            Coordinates coordinates,
            int age,
            long weight,
            DragonType type,
            DragonHead head) {

        // Creatable members:

        var uuid = java.util.UUID.randomUUID();
        id = uuid.getMostSignificantBits() ^ uuid.getLeastSignificantBits();
        creationDate = java.time.LocalDateTime.now();

        // Settable and checkable members:

        if (name == null) {
            throw new NullPointerException("Name cannot be null");
        }
        this.name = name;

        if (coordinates == null) {
            throw new NullPointerException("Coordinated cannot be null");
        }
        this.coordinates = coordinates;
```

```java
        if (age <= 0) {
            throw new IllegalArgumentException("Age must be greater than 0");
        }
        this.age = age;

        if (weight <= 0) {
            throw new IllegalArgumentException("Age must be greater than 0");
        }
        this.weight = weight;

        if (head == null) {
            throw new NullPointerException("Head cannot be null");
        }
        this.head = head;

        // Settable and not checkable members:

        this.type = type;
    }

    /**
     * Constructs a new Dragon with basic attributes, type and head set to null.
     *
     * @param name the name of the dragon (cannot be null or empty)
     * @param coordinates the coordinates of the dragon (cannot be null)
     * @param age the age of the dragon (must be greater than 0)
     * @param weight the weight of the dragon (must be greater than 0)
     * @throws BadDataException if data validation fails
     * @throws NullPointerException if required parameters are null
     */
    public Dragon(String name, Coordinates coordinates, int age, long weight)
            throws BadDataException, NullPointerException {
        this(name, coordinates, age, weight, null, null);
    }

    /**
     * Returns the unique identifier of the dragon.
     *
     * @return the ID
     */
    public long getId() {
        return id;
    }

    /**
     * Returns the name of the dragon.
     *
     * @return the name
     */
```

```java
    public String getName() {
        return name;
    }

    /**
     * Returns the coordinates of the dragon.
     *
     * @return the coordinates
     */
    public Coordinates getCoordinates() {
        return coordinates;
    }

    /**
     * Returns the creation date of the dragon.
     *
     * @return the creation date
     */
    public java.time.LocalDateTime getCreationDate() {
        return creationDate;
    }

    /**
     * Returns the age of the dragon.
     *
     * @return the age
     */
    public int getAge() {
        return age;
    }

    /**
     * Returns the weight of the dragon.
     *
     * @return the weight
     */
    public long getWeight() {
        return weight;
    }

    /**
     * Returns whether the dragon is speaking.
     *
     * @return true if the dragon is speaking, false otherwise
     */
    public boolean isSpeaking() {
        return speaking;
    }

    /**
```

```
     * Returns the type of the dragon.
     *
     * @return the dragon type
     */
    public DragonType getType() {
        return type;
    }

    /**
     * Returns the head of the dragon.
     *
     * @return the dragon head
     */
    public DragonHead getHead() {
        return head;
    }

    /**
     * Returns the hash code based on the dragon's ID.
     *
     * @return the hash code
     */
    @Override
    public int hashCode() {
        return id.hashCode();
    }

    /**
     * Returns true if equals object's classes and compareTo() returns 0.
     *
     * @param obj the other obj
     * @return true if objects equals
     */
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (getClass() != obj.getClass()) return false;
        return compareTo((Dragon) obj) == 0;
    }

    /**
     * Compares this dragon to another based on their IDs.
     *
     * @param other the other dragon to compare to
     * @return a negative integer, zero, or a positive integer as this dragon's
ID is less than,
     *     equal to, or greater than the specified dragon's ID
     */
    @Override
    public int compareTo(Dragon other) {
```

```java
        return id.compareTo(other.id);
    }

    /**
     * The unique identifier of the dragon. Cannot be null, must be greater than
0, must be unique,
     * and is generated automatically.
     */
    private final Long id;

    /** The name of the dragon. Cannot be null, string cannot be empty. */
    private final String name;

    /** The coordinates of the dragon. Cannot be null. */
    private Coordinates coordinates;

    /** The creation date of the dragon. Cannot be null, generated automatically.
*/
    private final java.time.LocalDateTime creationDate;

    /** The age of the dragon. Must be greater than 0. */
    private int age;

    /** The weight of the dragon. Must be greater than 0. */
    private long weight;

    /** Indicates whether the dragon is speaking. */
    private boolean speaking;

    /** The type of the dragon. Can be null. */
    private final DragonType type;

    /** The head of the dragon. */
    private final DragonHead head;
}
```

DragonHead

```java
package dragon;

/** Represents the head of a dragon, including it's size and tooth count. */
public class DragonHead implements java.io.Serializable {
    /**
     * Constructs a new DragonHead with the specified size and tooth count.
     *
     * @param size the size of the dragon's head
     * @param toothCount the number of teeth in the dragon's head
     */
    public DragonHead(float size, float toothCount) {
        this.size = size;
        this.toothCount = toothCount;
    }

    /**
     * Returns the size of the dragon's head.
     *
     * @return the size
     */
    public float getSize() {
        return size;
    }

    /**
     * Returns the number of teeth in the dragon's head.
     *
     * @return the tooth count
     */
    public float getToothCount() {
        return toothCount;
    }

    /** The size of the dragon head. */
    private float size;

    /** The number of teeth in the dragon head. */
    private float toothCount;
}
```

DragonType

```java
package dragon;

/**
 * Represents the different types of dragons in the system.
 */
```

```java
public enum DragonType {
    WATER,
    UNDERGROUND,
    AIR,
    FIRE;
}
```

package dragon.view

View

```java
package dragon.view;

import java.io.InputStream;
import java.io.OutputStream;
import java.util.stream.Stream;

import dragon.Dragon;

/**
 * Converts a stream of {@link Dragon} objects to/from an I/O representation.
 */
public interface View {

    /**
     * Encodes the given stream into an input stream representation.
     *
     * @param stream stream of dragons (may be null depending on implementation)
     * @return encoded input stream
     */
    InputStream toView(Stream<Dragon> stream);

    /**
     * Decodes dragons from the provided output stream (typically a buffer pro-
duced by storage).
     *
     * @param stream source buffer stream
     * @return decoded stream of dragons
     */
    Stream<Dragon> fromView(OutputStream stream);

}
```

JsonView

```
package dragon.view;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.OutputStream;
import java.util.stream.Stream;

import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

import dragon.Dragon;

/**
 * JSON-based {@link View} implementation using Gson.
 *
 * <p>Serializes/deserializes a stream of dragons as a JSON array.
 */
public class JsonView implements View {

    /** Gson instance configured for pretty printing. */
    private final Gson gson = new GsonBuilder().setPrettyPrinting().create();

    /**
     * Serializes a stream of dragons to JSON.
     *
     * @param stream stream of dragons (may be null)
     * @return input stream containing JSON array
     */
    @Override
    public InputStream toView(Stream<Dragon> stream) {
        if (stream == null) {
            return new ByteArrayIn-
putStream("[]".getBytes(core.Defaults.CHARSET));
        }

        var baos = new ByteArrayOutputStream();
        try (var writer = new java.io.OutputStreamWriter(baos,
core.Defaults.CHARSET);
                var jsonWriter = gson.newJsonWriter(writer)) {
            jsonWriter.beginArray();
            stream.forEach(dragon -> gson.toJson(dragon, Dragon.class, jsonWrit-
er));
            jsonWriter.endArray();
            jsonWriter.flush();
        } catch (java.io.IOException e) {
            // ByteArrayOutputStream does not throw on close, but we keep
            // this to satisfy the compile-time checked exception.
            throw new RuntimeException(e);
```

```
        }

        return new ByteArrayInputStream(baos.toByteArray());
    }

    /**
     * Deserializes dragons from a byte buffer.
     *
     * <p>Requires {@link ByteArrayOutputStream} as input container.
     *
     * @param stream output stream containing serialized JSON bytes
     * @return stream of decoded dragons
     * @throws IllegalArgumentException if the given stream is not a {@link
ByteArrayOutputStream}
     */
    @Override
    public Stream<Dragon> fromView(OutputStream stream) {
        if (!(stream instanceof ByteArrayOutputStream)) {
            throw new IllegalArgumentException("JsonView.fromView expects a
ByteArrayOutputStream");
        }

        byte[] bytes = ((ByteArrayOutputStream) stream).toByteArray();
        if (bytes == null || bytes.length == 0) {
            return Stream.empty();
        }

        try {

            var input = new ByteArrayInputStream(bytes);
            var reader = new java.io.InputStreamReader(input,
core.Defaults.CHARSET);
            var jsonReader = new com.google.gson.stream.JsonReader(reader);

            jsonReader.beginArray();
            // } catch (java.io.IOException e) {
            //     throw new RuntimeException(e);
            // }

            var iterator =
                    new java.util.Iterator<Dragon>() {
                        private Dragon next = null;
                        private boolean finished = false;

                        private void finishQuietly() {
                            finished = true;
                            try {
                                jsonReader.close();
                            } catch (java.io.IOException ignored) {
                            }
```

```
                        }

                        private void advance() {
                            if (finished || next != null) {
                                return;
                            }

                            try {
                                if (!jsonReader.hasNext()) {
                                    finished = true;
                                    jsonReader.endArray();
                                    jsonReader.close();
                                    return;
                                }
                                next = gson.fromJson(jsonReader, Dragon.class);
                            } catch (java.io.IOException e) {
                                throw new RuntimeException(e);
                            } catch (com.google.gson.JsonSyntaxException e) {
                                // Includes MalformedJsonException as a cause for
broken JSON.
                                System.err.println("Error parsing JSON (the data
is lost): " + e.getMessage());
                                finishQuietly();
                            } catch (com.google.gson.JsonIOException e) {
                                System.err.println("Error reading JSON (the data
is lost): " + e.getMessage());
                                finishQuietly();
                            } catch (com.google.gson.JsonParseException e) {
                                System.err.println("Error parsing JSON (the data
is lost): " + e.getMessage());
                                finishQuietly();
                            }
                        }

                        @Override
                        public boolean hasNext() {
                            advance();
                            return next != null;
                        }

                        @Override
                        public Dragon next() {
                            advance();
                            if (next == null) {
                                throw new java.util.NoSuchElementException();
                            }
                            var result = next;
                            next = null;
                            return result;
                        }
```

```
                    };

            var spliterator =
                    java.util.Spliterators.spliteratorUnknownSize(
                            iterator,
                            java.util.Spliterator.ORDERED | ja-
va.util.Spliterator.NONNULL);
            var streamResult = java.util.stream.StreamSupport.stream(spliterator,
false);
            streamResult =
                    streamResult.onClose(
                            () -> {
                                try {
                                    jsonReader.close();
                                } catch (java.io.IOException ignored) {
                                }
                            });
            return streamResult;
        } catch (java.io.IOException e) {
            return Stream.empty();
        } catch (com.google.gson.JsonIOException e) {
            System.err.println("Error reading JSON: " + e.getMessage());
            return Stream.empty();
        } catch (com.google.gson.JsonSyntaxException e) {
            System.err.println("Error parsing JSON: " + e.getMessage());
            return Stream.empty();
        } catch (com.google.gson.JsonParseException e) {
            System.err.println("Error parsing JSON: " + e.getMessage());
            return Stream.empty();
        }
    }
}
```

StringView

```
package dragon.view;

import java.io.InputStream;
import java.io.OutputStream;
import java.util.stream.Stream;

import dragon.Dragon;

/**
 * Human-readable text {@link View} implementation.
 *
 * <p>Formats each dragon as a multi-line block with basic indentation for nested
objects.
 */
public class StringView implements View {
```

```java
    /**
     * Formats a stream of dragons into a text representation.
     *
     * @param stream stream of dragons
     * @return input stream containing formatted text
     */
    @Override
    public InputStream toView(Stream<Dragon> stream) {
        StringBuilder sb = new StringBuilder();

        stream.forEach(dragon -> {
            // Root-level fields
            appendField(sb, "id:           ", String.valueOf(dragon.getId()), 0);
            appendField(sb, "name:         ", dragon.getName(), 0);
            appendField(sb, "creationDate: ",
String.valueOf(dragon.getCreationDate()), 0);
            appendField(sb, "age:          ", String.valueOf(dragon.getAge()),
0);
            appendField(sb, "weight:       ", String.valueOf(dragon.getWeight()),
0);
            appendField(sb, "speaking:     ",
String.valueOf(dragon.isSpeaking()), 0);

            // Nested fields
            appendField(sb, "coordinates:  ", null, 0);
            if (dragon.getCoordinates() != null) {
                appendField(sb, "x:            ",
String.valueOf(dragon.getCoordinates().getX()), 1);
                appendField(sb, "y:            ",
String.valueOf(dragon.getCoordinates().getY()), 1);
            }

            appendField(sb, "type:           ", dragon.getType() == null ? "null"
: dragon.getType().name(), 0);

            appendField(sb, "head:", null, 0);
            if (dragon.getHead() != null) {
                appendField(sb, "size:         ",
String.valueOf(dragon.getHead().getSize()), 1);
                appendField(sb, "toothCount:   ",
String.valueOf(dragon.getHead().getToothCount()), 1);
            }

            // Separator between dragons
            sb.append(System.lineSeparator());
        });

        return new ja-
va.io.ByteArrayInputStream(sb.toString().getBytes(core.Defaults.CHARSET));
```

```java
    }

    /**
     * Appends a single formatted field with indentation.
     *
     * @param sb target string builder
     * @param name field label (including alignment spaces if desired)
     * @param value value string or null to omit value after the label
     * @param indentLevel indentation level (each level adds two spaces)
     */
    private static void appendField(StringBuilder sb, String name, String value,
int indentLevel) {
        for (int i = 0; i < indentLevel; i++) {
            sb.append("  ");
        }

        sb.append(name);

        if (value != null) {
            sb.append(' ').append(value);
        }

        sb.append(System.lineSeparator());
    }

    /**
     * Parsing is not implemented for this view.
     *
     * @param stream ignored
     * @return empty stream
     */
    @Override
    public Stream<Dragon> fromView(OutputStream stream) {
        return Stream.empty();
    }

}
```

package collection

API

```
package collection;

import java.util.function.Predicate;
import java.util.stream.Stream;

import dragon.Dragon;

/**
 * Abstraction for working with a collection of {@link Dragon} entities.
 *
 * <p>Provides basic CRUD-like operations and stream-based querying.
 */
public interface API {
    /**
     * Adds a single element to the collection.
     *
     * @param element the element to add
     */
    void add(Dragon element);

    /**
     * Adds multiple elements to the collection.
     *
     * @param elements stream of elements to add
     */
    void add(Stream<Dragon> elements);

    /**
     * Updates an element identified by its id.
     *
     * <p>Typical behavior is to remove existing element with the same id and
then add the given one.
     *
     * @param element new element value (may be null depending on implementation)
     */
    void updateById(Dragon element);

    /** Removes all elements from the collection. */
    void clear();

    /**
     * Returns a sequential stream of elements in the collection.
     *
     * @return stream of elements
     */
    Stream<Dragon> getStream();

    /**
```

```
     * Removes all elements that match the given filter.
     *
     * @param filter predicate used to select elements to remove
     */
    void removeIf(Predicate<? super Dragon> filter);

    /**
     * Counts the number of elements that match the given filter.
     *
     * @param filter predicate used to select elements to count
     * @return number of matching elements
     */
    int countIf(Predicate<? super Dragon> filter);
}
```

Collection

```java
package collection;

import java.util.Collections;
import java.util.HashSet;
import java.util.Set;
import java.util.function.Predicate;
import java.util.stream.Collectors;
import java.util.stream.Stream;

import dragon.Dragon;

/**
 * In-memory implementation of {@link API} backed by a {@link HashSet}.
 */
public class Collection implements collection.API {
    /** Creates an empty collection. */
    public Collection() {
        this.buffer = new HashSet<>();
    }

    /**
     * Creates a collection initialized from a set.
     *
     * @param buffer initial elements
     */
    public Collection(Set<Dragon> buffer) {
        this.buffer = new HashSet<>(buffer);
    }

    /**
     * Creates a collection initialized from a stream.
     *
     * @param stream initial elements stream
     */
    public Collection(Stream<Dragon> stream) {
        this(stream.collect(Collectors.toSet()));
    }

    /**
     * Adds a single element to the set.
     *
     * @param element element to add
     */
    @Override
    public void add(Dragon element) {
        buffer.add(element);
    }

    /**
```

```java
     * Adds all elements from the provided stream.
     *
     * @param elements stream of elements to add
     */
    @Override
    public void add(Stream<Dragon> elements) {
        buffer.addAll(elements.collect(Collectors.toSet()));
    }

    /**
     * Replaces an element with the same id as the provided one.
     *
     * @param element replacement element
     */
    @Override
    public void updateById(Dragon element) {
        buffer.removeIf(e -> e.getId() == element.getId());
        if (element != null)
            buffer.add(element);
    }

    /**
     * Returns a stream view of the underlying set.
     *
     * @return stream of dragons
     */
    @Override
    public Stream<Dragon> getStream() {
        return buffer.stream();
    }

    /**
     * Removes all elements matching the predicate.
     *
     * @param filter predicate selecting elements to remove
     */
    @Override
    public void removeIf(Predicate<? super Dragon> filter) {
        buffer.removeIf(filter);
    }

    /**
     * Counts all elements matching the predicate.
     *
     * @param filter predicate selecting elements to count
     * @return number of matching elements
     */
    @Override
    public int countIf(Predicate<? super Dragon> filter) {
        int counter = 0;
```

```java
        for (var x : buffer)
            if (filter.test(x))
                counter++;

        return counter;
    }

    @Override
    public void clear() {
        buffer.clear();
    }

    /**
     * Returns an unmodifiable view of the underlying set.
     *
     * @return unmodifiable set of elements
     */
    public Set<Dragon> getBuffer() {
        return Collections.unmodifiableSet(buffer);
    }

    /** Backing storage for the collection. */
    private HashSet<Dragon> buffer;

}
```

package storage

Storage

```java
package storage;

import java.io.InputStream;
import java.io.OutputStream;
import java.util.Date;

/**
 * Persistence abstraction for storing and loading serialized data.
 */
public interface Storage {
    /**
     * Saves the provided input stream content to the storage destination.
     *
     * @param stream data to save
     */
    void save(InputStream stream);

    /**
     * Loads data from storage into an in-memory buffer.
     *
     * @return output stream containing loaded bytes
     */
    OutputStream load();

    /**
     * Returns the creation date of the underlying storage resource.
     *
     * @return creation date, or null if not available
     */
    Date getDateCreated();

    /**
     * Returns the last modification date of the underlying storage resource.
     *
     * @return last modification date
     */
    Date getDateModified();
}
```

FileStorage

```java
package storage;

import java.io.BufferedWriter;
import java.io.ByteArrayOutputStream;
import java.io.FileInputStream;
import java.io.FileWriter;
import java.io.IOException;
```

```java
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.attribute.FileTime;
import java.util.Date;

/**
 * File-based {@link Storage} implementation.
 *
 * <p>Stores the serialized collection (as bytes) in a single file. Metadata such
as creation and
 * modification time is obtained from the file system rather than being serial-
ized.
 */
public class FileStorage implements Storage {

    /**
     * Creates a file storage bound to the specified path.
     *
     * @param filename file path to read from and write to
     */
    public FileStorage(Path filename) {
        this.filename = filename;
    }

    /**
     * Writes the provided stream contents to the file.
     *
     * @param stream data to save
     */
    @Override
    public void save(InputStream stream) {
        try (var reader = new InputStreamReader(stream, core.Defaults.CHARSET);
                var writer = new BufferedWriter(new FileWrit-
er(filename.toFile()))) {
            reader.transferTo(writer);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /**
     * Loads the file contents into a {@link ByteArrayOutputStream}.
     *
     * @return output stream containing file bytes
     */
    @Override
```

```
    public OutputStream load() {

        var baos = new ByteArrayOutputStream();

        try (var reader = new InputStreamReader(new FileIn-
putStream(filename.toFile()));
                var writer = new OutputStreamWriter(baos, core.Defaults.CHARSET))
{
            reader.transferTo(writer);
        }
         catch (IOException e) {
            System.err.println("Error reading file (the data is lost): " +
e.getMessage());
        }
        return baos;
    }

    /**
     * Returns the file creation time as reported by the file system.
     *
     * @return creation date, or null if it cannot be determined
     */
    @Override
    public Date getDateCreated() {
        try {
            new Date(((FileTime) (Files.getAttribute(filename, "creation-
Time"))).toMillis());
        } catch (IOException e) {
            System.err.println("Error getting creation date: " + e.getMessage());
            }
        return null;
    }

    /**
     * Returns the file modification time.
     *
     * @return last modified date
     */
    @Override
    public Date getDateModified() {
        return new Date(filename.toFile().lastModified());
    }

    /** Backing file path. */
    private final Path filename;
}
```

ui

Command

```
package ui;

/**
 * A parsed user command with optional arguments.
 *
 * @param command command type
 * @param dragon optional dragon argument (for commands operating on a dragon)
 * @param type optional dragon type argument (for counting/filtering by type)
 * @param name optional name argument (for filtering by name prefix)
 */
public record Command(Commands command, dragon.Dragon dragon, dragon.DragonType
type, String name) {
}
```

Commands

```
package ui;

/**
 * Supported command list for the interactive UI.
 *
 * <p>Each command has a stable text form as entered by the user and a derived
integer code.
 */
public enum Commands {

    HELP("help"),
    INFO("info"),
    ADD("add"),
    SHOW("show"),
    UPDATE_BY_ID("update_by_id"),
    REMOVE_BY_ID("remove_by_id"),
    CLEAR("clear"),
    SAVE("save"),
    EXECUTE_SCRIPT("execute_script"),
    EXIT("exit"),
    REMOVE_GREATER("remove_greater"),
    REMOVE_LOWER("remove_lower"),
    HISTORY("history"),
    COUNT_BY_TYPE("count_by_type"),
    COUNT_GREATER_THAN_TYPE("count_greater_than_type"),
    FILTER_STARTS_WITH_NAME("filter_starts_with_name");

    private final int code;
    private final String text;

    /**
     * Creates a command descriptor.
```

```
     *
     * @param text command text as typed by the user
     */
    Commands(String text) {
        this.text = text;
        this.code = text.hashCode();
    }

    /**
     * Returns the integer command code.
     *
     * @return command code
     */
    public int getCode() {
        return code;
    }

    /**
     * Returns the command text.
     *
     * @return command text
     */
    public String getText() {
        return text;
    }

}
```

Executable

```
package ui;

/**
 * Functional interface representing a command handler.
 */
public interface Executable {

    /**
     * Executes a command using the provided UI context.
     *
     * @param command parsed command data
     * @param ui UI context
     */
    void execute(Command command, UI ui);

}
```

TextUIHandler

```
package ui;
```

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.Locale;
import java.util.Objects;
import java.util.function.Consumer;

/**
 * Interactive text UI handler.
 *
 * <p>Reads commands from stdin (or from script files via {@code execute_script})
and dispatches them
 * to {@link UI}. Supports a bounded script recursion depth and basic input vali-
dation for building
 * {@link dragon.Dragon} objects.
 */
public class TextUIHandler implements Runnable {
    /**
     * Creates the handler bound to a {@link UI} instance.
     *
     * @param ui UI facade used to execute commands
     * @throws NullPointerException if ui is null
     */
    public TextUIHandler(UI ui) {
        this.ui = Objects.requireNonNull(ui, "ui");
        this.readers = new ArrayDeque<>();
        this.openScripts = new ArrayDeque<>();
        this.in = new BufferedReader(new InputStreamReader(System.in));

        System.out.println("Welcome to the Dragon database!");
        System.out.println("You can manage dragons using the following commands
(type 'help' to list them):");

        readers.push(in);

        commandHandlers = new HashMap<>();
        registerHandlers();
    }

    /**
     * Main interactive loop.
     *
```

```
     * <p>Reads lines from the current input source, supports returning from
scripts, and executes
     * parsed commands until input is exhausted.
     */
    @Override
    public void run() {
        try {
            while (true) {
                printPromptIfInteractive();
                var line = readLineFromCurrentInput();
                if (line == null) {
                    if (!closeCurrentScriptIfAny()) {
                        return;
                    }
                    continue;
                }

                line = line.trim();
                if (line.isEmpty()) {
                    continue;
                }

                dispatchLine(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

    }

    /** UI facade used to execute parsed commands. */
    private UI ui;

    /** Primary reader for interactive stdin. */
    private BufferedReader in;

    /** Stack of active readers (stdin + nested script readers). */
    private final Deque<BufferedReader> readers;

    /** Stack of script paths currently being executed (for recursion detection).
*/
    private final Deque<Path> openScripts;

    /** Command dispatcher mapping from command text to handler. */
    private final HashMap<String, Consumer<String>> commandHandlers;

    /** Maximum allowed nested script execution depth. */
    private static final int MAX_SCRIPT_DEPTH = 16;

    /**
```

```java
     * Prints the prompt only when running in interactive mode (stdin is the only
source).
     */
    private void printPromptIfInteractive() {
        if (readers.size() == 1) {
            System.out.print("> ");
        }
    }

    /**
     * Reads a single line from the current input source.
     *
     * @return next line, or null on EOF
     * @throws IOException if reading fails
     */
    private String readLineFromCurrentInput() throws IOException {
        return readers.peek().readLine();
    }

    /**
     * Closes current script reader (if any) and returns to the previous input
source.
     *
     * @return true if a script reader was closed, false if currently in interac-
tive mode
     */
    private boolean closeCurrentScriptIfAny() {
        if (readers.size() <= 1) {
            return false;
        }

        var reader = readers.pop();
        try {
            reader.close();
        } catch (IOException ignored) {
        }

        var path = openScripts.pollFirst();
        if (path != null) {
            System.out.printf("Returning to previous input source (script fin-
ished): %s%n", path);
        }
        return true;
    }

    /**
     * Parses and dispatches a command line to its handler.
     *
     * @param line raw input line
     */
```

```java
    private void dispatchLine(String line) {
        var parts = line.split("\\s+", 2);
        var cmd = parts[0].toLowerCase(Locale.ROOT);
        var args = parts.length > 1 ? parts[1].trim() : "";

        var handler = commandHandlers.get(cmd);
        if (handler == null) {
            System.out.printf("Unknown command: %s%n", cmd);
            return;
        }

        try {
            handler.accept(args);
        } catch (RuntimeException e) {
            System.out.printf("Error while executing command '%s': %s%n", cmd,
e.getMessage());
        }
    }

    /**
     * Registers all supported command handlers.
     */
    private void registerHandlers() {
        commandHandlers.put(Commands.HELP.getText(), args -> printHelp());
        commandHandlers.put(Commands.EXIT.getText(), args -> System.exit(0));
        commandHandlers.put(Commands.EXECUTE_SCRIPT.getText(),
this::handleExecuteScript);

        commandHandlers.put(Commands.INFO.getText(), args -> ui.execute(new Com-
mand(Commands.INFO, null, null, null)));
        commandHandlers.put(Commands.SHOW.getText(), args -> ui.execute(new Com-
mand(Commands.SHOW, null, null, null)));

        commandHandlers.put(Commands.CLEAR.getText(), args -> {
            ui.execute(new Command(Commands.CLEAR, null, null, null));
            System.out.println("Collection cleared successfully.");
        });

        commandHandlers.put(Commands.SAVE.getText(), args -> {
            ui.execute(new Command(Commands.SAVE, null, null, null));
            System.out.println("Collection saved successfully.");
        });

        commandHandlers.put(Commands.HISTORY.getText(), args -> ui.execute(new
Command(Commands.HISTORY, null, null, null)));

        commandHandlers.put(Commands.ADD.getText(), args -> {
            ui.execute(new Command(Commands.ADD, readDragon(), null, null));
            System.out.println("Dragon added successfully.");
        });
```

```
        commandHandlers.put(Commands.REMOVE_GREATER.getText(), args -> {
            ui.execute(new Command(Commands.REMOVE_GREATER, readDragon(), null,
null));
            System.out.println("All dragons greater than the given one were re-
moved.");
        });

        commandHandlers.put(Commands.REMOVE_LOWER.getText(), args -> {
            ui.execute(new Command(Commands.REMOVE_LOWER, readDragon(), null,
null));
            System.out.println("All dragons lower than the given one were re-
moved.");
        });

        commandHandlers.put(Commands.REMOVE_BY_ID.getText(), args -> {
            var id = parseLongRequired(args, "id");
            ui.execute(new Command(Commands.REMOVE_BY_ID, dragonWithIdOnly(id),
null, null));
            System.out.println("Dragon removed by id.");
        });

        commandHandlers.put(Commands.UPDATE_BY_ID.getText(), args -> {
            var id = parseLongRequired(args, "id");
            var d = readDragon();
            forceSetDragonId(d, id);
            ui.execute(new Command(Commands.UPDATE_BY_ID, d, null, null));
            System.out.println("Dragon updated successfully.");
        });

        commandHandlers.put(Commands.COUNT_BY_TYPE.getText(), args -> {
            var type = parseDragonTypeRequired(args);
            ui.execute(new Command(Commands.COUNT_BY_TYPE, null, type, null));
        });

        commandHandlers.put(Commands.COUNT_GREATER_THAN_TYPE.getText(), args -> {
            var type = parseDragonTypeRequired(args);
            ui.execute(new Command(Commands.COUNT_GREATER_THAN_TYPE, null, type,
null));
        });

        commandHandlers.put(Commands.FILTER_STARTS_WITH_NAME.getText(), args -> {
            if (args == null || args.isBlank()) {
                throw new IllegalArgumentException("You must provide a name sub-
string.");
            }
            ui.execute(new Command(Commands.FILTER_STARTS_WITH_NAME, null, null,
args));
        });
    }
```

```java
    /**
     * Handles {@code execute_script} command by pushing a new reader onto the
stack.
     *
     * @param args path to the script file (may be quoted)
     * @throws IllegalArgumentException if the path is invalid, unreadable, or
recursion is detected
     */
    private void handleExecuteScript(String args) {
        if (args == null || args.isBlank()) {
            throw new IllegalArgumentException("You must provide a script file
name.");
        }

        if (readers.size() - 1 >= MAX_SCRIPT_DEPTH) {
            System.out.printf("Maximum script recursion depth reached: %d%n",
MAX_SCRIPT_DEPTH);
            return;
        }

        var raw = args.trim();
        raw = stripMatchingQuotes(raw);

        Path path;
        try {
            var p = Paths.get(raw);
            if (!p.isAbsolute()) {
                p = getScriptBaseDir().resolve(p);
            }
            path = p.toAbsolutePath().normalize();
        } catch (Exception e) {
            throw new IllegalArgumentException("Invalid script file path.");
        }

        if (!Files.exists(path) || !Files.isRegularFile(path) ||
!Files.isReadable(path)) {
            throw new IllegalArgumentException("Script file not found or not
readable: " + path);
        }

        if (openScripts.contains(path)) {
            throw new IllegalArgumentException("Recursion detected: script is al-
ready being executed: " + path);
        }

        try {
            var br = new BufferedReader(new FileReader(path.toFile()));
            readers.push(br);
            openScripts.push(path);
```

```
            System.out.printf("Executing script: %s%n", path);
        } catch (IOException e) {
            throw new IllegalArgumentException("Failed to open script: " + path);
        }
    }

    /**
     * Returns a base directory for resolving relative script paths.
     *
     * <p>If scripts are nested, relative paths are resolved against the directo-
ry of the current script.
     *
     * @return base directory path
     */
    private Path getScriptBaseDir() {
        if (!openScripts.isEmpty()) {
            var current = openScripts.peek();
            var parent = current != null ? current.getParent() : null;
            if (parent != null) {
                return parent;
            }
        }
        return Paths.get("").toAbsolutePath();
    }

    /**
     * Removes matching leading and trailing quotes (single or double) from a
string.
     *
     * @param s input string
     * @return unquoted string
     */
    private String stripMatchingQuotes(String s) {
        if (s == null) return null;
        if (s.length() >= 2) {
            var first = s.charAt(0);
            var last = s.charAt(s.length() - 1);
            if ((first == '"' && last == '"') || (first == '\'' && last == '\''))
{
                return s.substring(1, s.length() - 1).trim();
            }
        }
        return s;
    }

    /**
     * Prints available command names.
     */
    private void printHelp() {
        System.out.println("Available commands:");
```

```
        for (var c : Commands.values()) {
            System.out.println("- " + c.getText());
        }
    }

    /**
     * Parses a required long value.
     *
     * @param s raw string
     * @param argName argument name for error messages
     * @return parsed long
     * @throws IllegalArgumentException if missing or invalid
     */
    private long parseLongRequired(String s, String argName) {
        if (s == null || s.isBlank()) {
            throw new IllegalArgumentException("You must provide " + argName +
".");
        }
        try {
            return Long.parseLong(s.trim());
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Invalid number: " + s.trim());
        }
    }

    /**
     * Parses a required {@link dragon.DragonType} value.
     *
     * @param s raw string
     * @return parsed type
     * @throws IllegalArgumentException if missing or invalid
     */
    private dragon.DragonType parseDragonTypeRequired(String s) {
        if (s == null || s.isBlank()) {
            throw new IllegalArgumentException("You must provide type (one of:
WATER, UNDERGROUND, AIR, FIRE).");
        }
        try {
            return dragon.DragonType.valueOf(s.trim().toUpperCase(Locale.ROOT));
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Invalid type. Allowed: WATER, UN-
DERGROUND, AIR, FIRE.");
        }
    }

    /**
     * Reads an optional {@link dragon.DragonType} from input. Empty line is
treated as null.
     *
     * @return parsed type or null
```

```
     * @throws IOException on input error
     */
    private dragon.DragonType readDragonTypeNullable() throws IOException {
        while (true) {
            System.out.print("Enter type (WATER/UNDERGROUND/AIR/FIRE) or empty
line for null: ");
            var line = readLineFromCurrentInput();
            if (line == null) {
                throw new IllegalStateException("Unexpected end of input while
reading type.");
            }
            line = line.trim();
            if (line.isEmpty()) {
                return null;
            }
            try {
                return dragon.DragonType.valueOf(line.toUpperCase(Locale.ROOT));
            } catch (IllegalArgumentException e) {
                System.out.println("Error: invalid enum value. Please try
again.");
            }
        }
    }

    /**
     * Reads a non-empty string from input.
     *
     * @param prompt prompt to display
     * @return non-empty user input
     * @throws IOException on input error
     */
    private String readNonEmptyString(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            var line = readLineFromCurrentInput();
            if (line == null) {
                throw new IllegalStateException("Unexpected end of input.");
            }
            line = line.trim();
            if (!line.isEmpty()) {
                return line;
            }
            System.out.println("Error: value cannot be empty. Please try
again.");
        }
    }

    /**
     * Reads a float from input.
     *
```

```
     * @param prompt prompt to display
     * @return parsed float value
     * @throws IOException on input error
     */
    private float readFloat(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            var line = readLineFromCurrentInput();
            if (line == null) {
                throw new IllegalStateException("Unexpected end of input.");
            }
            line = line.trim();
            try {
                return Float.parseFloat(line);
            } catch (NumberFormatException e) {
                System.out.println("Error: you must enter a float number. Please
try again.");
            }
        }
    }

    /**
     * Reads a positive integer from input.
     *
     * @param prompt prompt to display
     * @return parsed integer greater than 0
     * @throws IOException on input error
     */
    private int readIntPositive(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            var line = readLineFromCurrentInput();
            if (line == null) {
                throw new IllegalStateException("Unexpected end of input.");
            }
            line = line.trim();
            try {
                var v = Integer.parseInt(line);
                if (v <= 0) {
                    System.out.println("Error: number must be > 0. Please try
again.");
                    continue;
                }
                return v;
            } catch (NumberFormatException e) {
                System.out.println("Error: you must enter an integer number.
Please try again.");
            }
        }
    }
```

```java
    /**
     * Reads a positive long from input.
     *
     * @param prompt prompt to display
     * @return parsed long greater than 0
     * @throws IOException on input error
     */
    private long readLongPositive(String prompt) throws IOException {
        while (true) {
            System.out.print(prompt);
            var line = readLineFromCurrentInput();
            if (line == null) {
                throw new IllegalStateException("Unexpected end of input.");
            }
            line = line.trim();
            try {
                var v = Long.parseLong(line);
                if (v <= 0) {
                    System.out.println("Error: number must be > 0. Please try
again.");
                    continue;
                }
                return v;
            } catch (NumberFormatException e) {
                System.out.println("Error: you must enter a long number. Please
try again.");
            }
        }
    }

    /**
     * Reads a {@link dragon.Dragon} from input by prompting for its fields.
     *
     * @return constructed dragon
     * @throws IllegalStateException on I/O error
     */
    private dragon.Dragon readDragon() {
        try {
            var name = readNonEmptyString("Enter name: ");

            core.Coordinates coordinates;
            while (true) {
                try {
                    var x = readFloat("Enter coordinates.x (float, > -359): ");
                    var y = readFloat("Enter coordinates.y (float, <= 603): ");
                    coordinates = new core.Coordinates(x, y);
                    break;
                } catch (core.BadDataException | NullPointerException e) {
                    System.out.printf("Coordinates error: %s%n", e.getMessage());
```

```
                }
            }

            var age = readIntPositive("Enter age (int, > 0): ");
            var weight = readLongPositive("Enter weight (long, > 0): ");
            var type = readDragonTypeNullable();

            var headSize = readFloat("Enter head.size (float): ");
            var headTeeth = readFloat("Enter head.toothCount (float): ");
            var head = new dragon.DragonHead(headSize, headTeeth);

            return new dragon.Dragon(name, coordinates, age, weight, type, head);
        } catch (IOException e) {
            throw new IllegalStateException("Input error: " + e.getMessage());
        }
    }

    /**
     * Creates a temporary dragon and overwrites its id using reflection.
     *
     * <p>Used for commands that operate by id without requiring full entity in-
put.
     *
     * @param id target id
     * @return dragon instance with forced id
     */
    private dragon.Dragon dragonWithIdOnly(long id) {
        try {
            var d = new dragon.Dragon(
                    "stub",
                    new core.Coordinates(-358f, 0f),
                    1,
                    1L,
                    null,
                    new dragon.DragonHead(1f, 1f));
            forceSetDragonId(d, id);
            return d;
        } catch (Exception e) {
            throw new IllegalStateException("Не удалось создать временный объект
для id");
        }
    }

    /**
     * Sets the {@code id} field of a dragon instance via reflection.
     *
     * @param d dragon instance
     * @param id id to assign
     * @throws IllegalStateException if reflection fails
     */
```

```
    private void forceSetDragonId(dragon.Dragon d, long id) {
        try {
            var f = dragon.Dragon.class.getDeclaredField("id");
            f.setAccessible(true);
            f.set(d, id);
        } catch (ReflectiveOperationException e) {
            throw new IllegalStateException("Невозможно установить id у Dragon
(reflection): " + e.getMessage());
        }
    }
}
```

UI

```
package ui;

import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;

import dragon.view.JsonView;
import dragon.view.StringView;
import dragon.view.View;
import storage.Storage;

/**
 * Application facade that wires together collection, storage and command execu-
tion.
 *
 * <p>Contains a registry of supported commands and their implementations and
maintains a history
 * of recently executed commands.
 */
public class UI {

    /**
     * Creates a UI instance with explicitly provided views.
     *
     * @param collection collection API implementation
     * @param storage persistence implementation
     * @param out output writer for user-visible messages
     * @param storageView view used to serialize collection into storage format
     * @param outView view used to render collection to user-visible format
     */
    public UI(collection.API collection, Storage storage, OutputStreamWriter out,
View storageView, View outView) {
        this.collection = collection;
        this.storage = storage;
        this.out = out;

        this.storageView = storageView;
        this.outView = outView;
    }

    /**
     * Creates a UI instance with default views: JSON for storage and text for
output.
     *
     * @param collection collection API implementation
     * @param storage persistence implementation
```

```java
     * @param out output writer for user-visible messages
     */
    public UI(collection.API collection, Storage storage, OutputStreamWriter out)
{
        this(collection, storage, out, new JsonView(), new StringView());
    }

    /**
     * Executes a command using the command registry and records it in history.
     *
     * @param command command to execute
     */
    public void execute(Command command) {
        var execute = executes.get(command.command().getCode());
        execute.execute(command, this);
        addToHistory(command);
    }

    /**
     * Adds the command to the fixed-size history and flushes output.
     *
     * @param command command to record
     */
    private void addToHistory(Command command) {
        if (history.size() == 8) {
            history.removeFirst();
        }
        history.addLast(command);
        try {
            out.flush();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    /** Collection API used by commands. */
    private collection.API collection;

    /** Storage implementation used for save/load operations. */
    private Storage storage;

    /** Output destination for user-visible messages. */
    private OutputStreamWriter out;

    /** View used to serialize the collection for storage. */
    private View storageView;

    /** View used to render output for the user. */
    private View outView;
```

```java
    /** Circular history buffer of last 8 commands. */
    private final Deque<Command> history = new ArrayDeque<>(8);

    /** Mapping from command code to executable implementation. */
    private static final HashMap<Integer, Executable> executes;

    static {
        executes = new HashMap();

        executes.put(Commands.ADD.getCode(), (command, ui) ->
ui.collection.add(command.dragon()));
        executes.put(Commands.UPDATE_BY_ID.getCode(), (command, ui) ->
ui.collection.updateById(command.dragon()));
        executes.put(Commands.REMOVE_BY_ID.getCode(),
                (command, ui) -> ui.collection.removeIf(x -> x.getId() == com-
mand.dragon().getId()));
        executes.put(Commands.CLEAR.getCode(), (command, ui) ->
ui.collection.clear());
        executes.put(Commands.SAVE.getCode(),
                (command, ui) ->
ui.storage.save(ui.storageView.toView(ui.collection.getStream())));
        executes.put(Commands.REMOVE_GREATER.getCode(),
                (command, ui) -> ui.collection.removeIf(x ->
x.compareTo(command.dragon()) > 0));
        executes.put(Commands.REMOVE_LOWER.getCode(),
                (command, ui) -> ui.collection.removeIf(x ->
x.compareTo(command.dragon()) < 0));
        executes.put(Commands.COUNT_BY_TYPE.getCode(),
                (command, ui) -> {
                    try {
                        ui.out
                                .write(Integer
                                        .valueOf(ui.collection.countIf(x ->
x.getType() == command.type()))
                                        .toString() + "\n");
                    } catch (IOException e) {
                        e.printStackTrace();
                    }
                });
        executes.put(Commands.COUNT_GREATER_THAN_TYPE.getCode(),
                (command, ui) -> {
                    try {
                        ui.out.write(
                                Integer.valueOf(
                                        ui.collection
                                                .countIf(x ->
x.getType().compareTo(command.type()) > 0))
                                        .toString() + "\n");
                    } catch (IOException e) {
                        e.printStackTrace();
```

```
                    }

                });

        executes.put(Commands.FILTER_STARTS_WITH_NAME.getCode(),
                (command, ui) -> {
                    try (var isr = new InputStreamReader(ui.outView
                            .toView(ui.collection.getStream().filter(e ->
e.getName().startsWith(command.name()))))) {
                        isr.transferTo(ui.out);

                    } catch (IOException e) {
                        e.printStackTrace();
                    }
                });
        executes.put(Commands.SHOW.getCode(), (command, ui) -> {
            try (var isr = new InputStreamRead-
er(ui.outView.toView(ui.collection.getStream()))) {
                isr.transferTo(ui.out);

            } catch (IOException e) {
                e.printStackTrace();
            }
        });

        executes.put(Commands.INFO.getCode(), (command, ui) -> {
            var str = new StringBuilder();
            str.append("Collection type: HashSet<Dragon>\n");
            str.append("Date created: ");
            var date = ui.storage.getDateCreated();
            str.append(date != null ? date.toString() : "null");
            str.append("\nDate modified: ");
            var dateModified = ui.storage.getDateModified();
            str.append(dateModified != null ? dateModified.toString() : "null");
            str.append("\nNumber of elements: ");
            str.append(ui.collection.countIf(e -> true));
            str.append("\n");

            try {
                ui.out.write(str.toString());
            } catch (IOException e) {
                e.printStackTrace();
            }
        });

        executes.put(Commands.HISTORY.getCode(), (command, ui) -> {
            ui.history.stream().forEach(e -> {
                try {
                    ui.out.write(e.command().getText());
                    ui.out.write("\n");
```

```
                } catch (IOException exception) {
                    exception.printStackTrace();
                }
            });
        });

    }

}
```